

Eigene Kategorien, Erfolge, Titel und vollständige Erfolgsseiten

Public API 1.4

Erweiterungs-Addons können neben normalen leeren Tabs nun vollständige, datenbankgestützte Erfolgs-Tabs registrieren. Diese verwenden dieselbe native Oberfläche wie der Tab „Erfolge“: Kategorienmenü, Zusammenfassung, Gesamtfortschritt, tab-eigene Punkte, bis zu zehn Kategorie-Fortschrittsbalken, vier neueste Erfolge, Suche, Filter, Detailansicht und eigene Heldentaten.

Verbindliche Speichergrenze

Jedes Erweiterungspack muss eigene SavedVariables deklarieren. Definitionsdaten liegen nur zur Laufzeit vor; Fortschritt bleibt ausschließlich im Speicher des Erweiterungspacks. CA_LocalData, CA_Settings und andere Hauptaddon-SavedVariables dürfen nicht verwendet werden.

Kompatibilität

API 1.4 ist innerhalb Major-Version 1 abwärtskompatibel. Bestehende RegisterTab-, Kategorie-, Erfolgs-, Kriterien-, Titel- und Summary-Balken-Integrationen bleiben gültig. Neue Funktionen werden nur benötigt, wenn strukturierte leere Tabs oder vollständige eigene Erfolgsseiten gewünscht sind.

Inhalt

Kapitel	Thema
1-4	Schnellstart, Version, Module, Speicher und Registrierung
5-8	IDs, Kategorien, Erfolge, Kriterien, Titel und Trigger
9	Balken in der Zusammenfassung des Haupttabs
10-12	Tab-System, strukturierte leere Tabs und vollständige Erfolgs-Tabs
13-14	Ereignisse, Abfragen und Snapshots
15-16	Vollständiges Beispiel, Limits und Veröffentlichungsregeln

1. Schnellstart

1.1 Beispiel-TOC

```
## Interface: 11508, 20506
## Title: AnniversaryAchievements - ExamplePack
## Version: 1.0.0
## Author: ExampleAuthor
## Dependencies: AnniversaryAchievements
## SavedVariablesPerCharacter: ExamplePackDB

ExamplePack.lua
```

1.2 Modul registrieren

```
ExamplePackDB = ExamplePackDB or {}

local API = AnniversaryAchievementsAPI
assert(API and API:IsCompatible(1, 4),
    "AnniversaryAchievements Public API 1.4 required")

local Pack = API:RegisterModule("ExampleAuthor.ExamplePack", {
    name = "ExamplePack",
    version = "1.0.0",
    author = "ExampleAuthor",
    storage = ExamplePackDB,
})
```

Die Modul-ID ist der dauerhaft stabile technische Namensraum. Name, Version und Autor werden in den Einstellungen des Hauptaddons angezeigt. Bei erneuter Registrierung derselben Modul-ID wird derselbe Handle zurückgegeben, sofern dieselbe Speichertabelle verwendet wird.

1.3 Größere Definitionsbestände bündeln

```
Pack:BeginBulkRegistration()
-- Kategorien, Tabs, Erfolge und Kriterien registrieren
Pack:EndBulkRegistration()
```

Registrierungen werden automatisch in begrenzten Schritten finalisiert. Der explizite Bulk-Block ist empfohlen, wenn Definitionen über mehrere Funktionen oder Dateien verteilt werden. Pack:IsRegistrationPending() zeigt an, ob noch Arbeit aussteht.

2. Version, Client und Kompatibilität

Abfrage	Rückgabe
API.API_VERSION / API:GetVersion()	1.4.0 beziehungsweise 1, 4, 0
API:GetAddonVersion()	Installierte Addon-Version, in diesem Stand v2.3.37
API:IsCompatible(major, minimumMinor)	true nur bei gleichem Major und ausreichendem Minor
API:GetClientFlavor()	CLASSIC_ERA oder TBC_ANNIVERSARY
API:GetClientInterfaceVersion()	Aktive numerische Interface-Version
API:IsClassicEra() / API:IsTBCAnniversary()	Bequeme Flavor-Prüfungen

```
assert(API:IsCompatible(1, 4), "Public API 1.4 required")

if API:IsTBCAnniversary() then
    -- TBC-spezifische Definitionen registrieren
end
```

Versionsregel

Innerhalb Major-Version 1 bleiben vorhandene Funktionen kompatibel. Ein Pack, das API 1.4 benötigt, muss IsCompatible(1, 4) prüfen. Der Patchwert dient Fehlerkorrekturen und ist keine Feature-Grenze.

3. Module, Speicher und CPU-sichere Finalisierung

3.1 Eigene SavedVariables sind Pflicht

metadata.storage muss die eigene SavedVariables-Tabelle des Erweiterungspacks sein. Die API legt darin AnniversaryAchievements.schemaVersion und AnniversaryAchievements.modules[moduleID].achievements an. Eine Hauptaddon-Tabelle oder eine darin verschachtelte Tabelle wird abgewiesen.

```
-- Vom Erweiterungs-Addon deklarierte SavedVariable:
ExamplePackDB = ExamplePackDB or {}
```

```
local Pack = API:RegisterModule("ExampleAuthor.ExamplePack", {
    storage = ExamplePackDB,
    name = "ExamplePack",
    version = "1.0.0",
})
```

3.2 Finalisierungsmodell

Eigenschaft	Vertrag
Prüfungen je Schritt	Höchstens 20 Abschlussprüfungen
Zeitbudget je Schritt	Höchstens etwa 2 ms, sofern Profilzeit verfügbar ist
Datenbankereignisse	Pro Modul und Phase gebündelt
UI-Aktualisierung	Gebündelt; RequestUIRefresh löst keinen unmittelbaren Vollscan aus
Abschlussmeldung	REGISTRATION_COMMITTED nach vollständiger Finalisierung

4. Handles, Schlüssel und stabile IDs

Alle Registrierungsfunktionen verwenden modulinterne String-Schlüssel. Die API erzeugt deterministische numerische IDs in reservierten Bereichen. Zurückgegebene Handles sind schreibgeschützt und können in nachfolgenden Registrierungen direkt verwendet werden.

Handle	Wichtige Felder
Kategorie	kind, id, key, moduleID, tabID
Erfolg	kind, id, key, moduleID, categoryID, criterialIDs
Leerer Tab	kind="tab", key, moduleID, tabType="empty"
Erfolgs-Tab	kind="achievementTab", key, moduleID, tabType="achievements", tabID
Leere Tab-Kategorie	kind="tabCategory", key, moduleID, tabKey

ID-Bereich	Konstante
Kategorien	API.ID_RANGE.CATEGORY_FIRST bis CATEGORY_LAST
Erfolge	API.ID_RANGE.ACHIEVEMENT_FIRST bis ACHIEVEMENT_LAST
Kriterien	API.ID_RANGE.CRITERIA_FIRST bis CRITERIA_LAST
Eigene Kriterientypen	API.ID_RANGE.CRITERIA_TYPE_FIRST bis CRITERIA_TYPE_LAST

5. Kategorien und Unterkategorien

RegisterCategory erstellt eine echte Achievement-Kategorie. Ohne tab wird sie im normalen Spieler-Tab registriert. Für einen API-Erfolgs-Tab wird dessen Handle oder Schlüssel über tab angegeben.

```
local Root = Pack:RegisterCategory("adventure", {
    name = "Abenteuer",
})

local Child = Pack:RegisterCategory("treasures", {
    name = "Schätze",
    parent = Root,
})
```

Feld	Typ	Bedeutung
name	string, Pflicht	Sichtbarer Kategorienname
tab	Erfolgs-Tab-Handle/-Schlüssel oder nil	Zieltab; nil bedeutet normaler Spieler-Tab
parent	Kategorie-Handle/-Schlüssel oder nil	Unterkategorie im selben Tab
id	positive Ganzzahl, optional	Explizite stabile ID innerhalb des reservierten Bereichs
unavailable	boolean, optional	Kategorie und ihre Erfolge deaktivieren
order	number, optional	Sortierung im Kategorienmenü; Standard 100, Heldentaten standardmäßig zuletzt
featsOfStrength	boolean, optional	Nur in API-Erfolgs-Tabs: eigene Heldentaten-Hauptkategorie

Heldentaten

Die native Heldentaten-Kategorie des Haupttabs bleibt geschützt. Eigene Heldentaten sind ausschließlich in einem API-Erfolgs-Tab erlaubt und werden dort mit featsOfStrength=true gekennzeichnet.

6. Erfolge und Kriterien

```
local TreasureHunter = Pack:RegisterAchievement("treasure_hunter", {
    category = Child,
    name = "Schatzjäger",
    description = "Findet 25 besondere Schätze.",
    points = 10,
    icon = "-INV_Misc_QuestionMark",
    criteria = {
        {
            key = "found",
            name = "Besondere Schätze gefunden",
            type = "SPECIAL",
            data = { "treasure" },
            quantity = 25,
        },
    },
})
```

Achievement-Feld	Bedeutung
category	Kategorie-Handle, Schlüssel oder numerische ID
name / description / points / icon	Darstellung und Punktwert
criteria	Nicht leeres Array von Kriteriumsdefinitionen
anyCriteria	true: ein Kriterium genügt; sonst müssen alle erfüllt sein
previous	Vorheriger Erfolg derselben Erweiterung für Fortschrittsketten
rewardText / titleReward	Normale Belohnungszeile und optionale Titelbelohnung
hideCriteriaUI	Alle Kriterien nur in der UI ausblenden
faction / unavailable / priority	Fraktion, Verfügbarkeit und Sortierpriorität

Kriteriums-Feld	Bedeutung
key	Stabiler Schlüssel innerhalb des Erfolgs
type	API.CRITERIA_TYPE, Name eines Kerntyps oder eigener Typ
data	Typabhängiges Datenarray
quantity	Optionaler Zielwert; ohne quantity ist das Kriterium einmalig
name	Sichtbarer Text oder nil
hidden	Nur dieses Kriterium in der Standard-UI ausblenden
quantityFormatter	Optionale Funktion für die sichtbare Mengenanzeige

Erfolge dürfen im normalen Spieler-Tab und in API-Erfolgs-Tabs liegen. Die native Heldentaten-Kategorie des Haupttabs und ihre Nachfahren bleiben für Erweiterungen gesperrt.

7. Titelbelohnungen

```
Pack:RegisterAchievement("guild_legend", {
    category = Legends,
    name = "Gildenlegende",
    description = "Vollbringt eine legendäre Gildentat.",
    points = 10,
    icon = 134400,
    rewardText = "Gildenlegende",
    titleReward = {
        key = "guild_legend",
        format = "%s, die Gildenlegende",
        femaleFormat = "%s, die Gildenlegende",
    },
    criteria = {
        { key = "done", type = "SPECIAL", data = { "legend" } },
    },
})
```

Feld	Semantik
titleReward=true	Sichere Standarddefinition aus rewardText erzeugen
key	Wird als extension.<moduleID>.<key> namespaced
format / femaleFormat	Namensformat mit %s als Charakternamen-Platzhalter
nativeMask / nativeMaskFemale	Optional; verwendet nur einen serverseitig bereits bekannten Titel

API:GetEarnedTitles() und API:GetSelectedTitle() liefern Kopien. API.EVENT.TITLE_CHANGED meldet Änderungen der lokalen Titelauswahl. Eine native Maske gewährt niemals einen serverseitigen Titel.

8. Kriterientypen, Trigger, Metaerfolge und Reset

8.1 Eigenen Kriterientyp registrieren

```
local OPEN_GUILD_CACHE = Pack:RegisterCriteriaType(
    "OPEN_GUILD_CACHE", 1
)

Pack:Trigger(OPEN_GUILD_CACHE, { 1 }, 1, "increment")
Pack:Trigger(OPEN_GUILD_CACHE, { 1 }, 10, "set")
```

dataLength darf 0 bis 16 betragen. Trigger aktualisiert ausschließlich Kriterien, die dem auslösenden Erweiterungsmodul gehören. mode ist increment, set oder nil.

8.2 Metaerfolg

```
Pack:RegisterAchievement("meta", {
    category = Root,
    name = "Alles geschafft",
    description = "Schließt Schatzjäger ab.",
    points = 10,
    icon = 134400,
    criteria = {
        { key = "treasure", type = "COMPLETE_ACHIEVEMENT",
          data = { TreasureHunter } },
    },
})
```

Die API indiziert Abhängigkeiten. Beim Abschluss eines Erfolgs werden nur tatsächlich abhängige Metaerfolge erneut geprüft. Pack:ResetProgress() setzt ausschließlich den persistenten Fortschritt des eigenen Moduls zurück.

9. Zusätzliche Balken in der Haupttab-Zusammenfassung

```
Pack:RegisterSummaryProgressBar("pack_progress", {
    label = "ExamplePack",
    category = Root,
    order = 100,
    tooltip = "Abgeschlossene ExamplePack-Erfolge",
})
```

RegisterSummaryProgressBar ergänzt weiterhin die Zusammenfassung des normalen Haupttabs „Erfolge“. Es gibt höchstens vier globale Erweiterungsbalken. Alternativ zu category kann getProgress current, maximum und optional einen Text liefern; onClick bleibt für globale Balken frei definierbar.

Nicht verwechseln

RegisterSummaryProgressBar betrifft den normalen Haupttab. RegisterTabProgressBar gehört zu einem konkreten leeren oder vollständigen Erfolgs-Tab und navigiert beim Klick immer in die gleichzeitig erzeugte Hauptkategorie.

10. Das Tab-System in API 1.4

Die Anzahl zusätzlicher Tabs ist API-seitig nicht begrenzt. API 1.4 unterscheidet zwei Tab-Typen: leere Tabs für frei gestaltete Inhalte und vollständige Erfolgs-Tabs mit nativer Achievement-Funktionalität.

Funktion	Tab-Typ	Ergebnis
Pack:RegisterTab(key, definition)	empty	Freie Seite; optional mit strukturiertem Kategorienmenü und Balken
Pack:RegisterAchievementTab(key, definition)	achievements	Vollständige eigene Erfolgsseite auf derselben UI-Basis wie „Erfolge“
Pack:RegisterTabCategory(tab, key, definition)	empty	Frei gestaltbare Kategorie im linken Menü
Pack:RegisterCategory(key, {tab=...})	achievements	Echte Achievement-Kategorie im eigenen Erfolgs-Tab
Pack:RegisterTabProgressBar(tab, key, definition)	beide	Balken und Hauptkategorie als untrennbare Einheit

Gemeinsames Tab-Feld	Vertrag
name	Beschriftung des unteren Tab-Buttons
title	Überschrift oben; Standard ist name
icon	Optionale Textur oder File-ID
width	80 bis 160 Pixel; Standard 115
order	Sortierung über alle Erweiterungsmodule
tooltip	String oder geschützte Callback-Funktion
onShow / onHide / onRefresh	Geschützte Lebenszyklus-Callbacks

11. Strukturierte leere Tabs

11.1 Legacy-/Freiflächenmodus

```
local InfoTab = Pack:RegisterTab("info", {
    name = "Gilde",
    title = "Gildenübersicht",
    createFrame = function(module, parent)
        local text = parent:CreateFontString(nil, "OVERLAY",
            "GameFontHighlight")
        text:SetPoint("TOPLEFT", 20, -20)
        text:SetText("Eigener Inhalt")
    end,
})
```

Ohne registrierte Tab-Kategorien oder Tab-Balken bleibt der bisherige freie Vollflächenmodus erhalten. createFrame ist in API 1.4 optional; ein Tab kann zunächst leer registriert und anschließend strukturiert werden.

11.2 Beliebige viele Kategorien

```
local Members = Pack:RegisterTabCategory(InfoTab, "members", {
    name = "Mitglieder",
    createFrame = function(module, parent)
        -- Inhalt dieser Kategorie einmalig erzeugen
    end,
})

Pack:RegisterTabCategory(InfoTab, "officers", {
    name = "Offiziere",
    parent = Members,
    createFrame = function(module, parent) end,
})
```

Leere Tabs haben keine API-seitige Kategorienbegrenzung. Haupt- und Unterkategorien werden in einem scrollbaren linken Menü angezeigt. Jede Kategorie besitzt createFrame sowie optional onShow, onHide, onRefresh, tooltip und order.

11.3 Bis zu zehn Balken; jeder Balken erzeugt eine Hauptkategorie

```
local RaidProgress = Pack:RegisterTabProgressBar(InfoTab,
  "raid_progress", {
    name = "Raidfortschritt",      -- Name der Hauptkategorie
    label = "Raids",              -- optionaler Balkentext
    getProgress = function(module, context)
      return 3, 8, "3/8"
    end,
    createFrame = function(module, parent)
      -- Inhalt der Zielkategorie
    end,
    tooltip = "Aktueller Gildenfortschritt",
    categoryTooltip = "Raidübersicht öffnen",
  })
```

Der zurückgegebene Handle ist die gleichzeitig erzeugte Hauptkategorie. Beim Klick auf den Balken öffnet die Oberfläche immer diese Kategorie. Ein eigener onClick-Callback ist deshalb nicht zulässig. Ein leerer Tab kann höchstens zehn Balken, aber beliebig viele zusätzliche Kategorien besitzen.

12. Vollständige eigene Erfolgs-Tabs

12.1 Tab registrieren

```
local GuildAchievements = Pack:RegisterAchievementTab(
  "guild_achievements", {
    name = "Gilde",
    title = "Gildenerfolge",
    icon = "Interface\\Icons\\INV_BannerPVP_02",
    width = 115,
    order = 100,
  })
```

Native Funktionsgleichheit

Ein API-Erfolgs-Tab nutzt die echte Achievement-Datenbank und dieselben Oberflächenpfade wie „Erfolge“. Automatisch enthalten sind: Kategorienmenü, Summary-Seite, Gesamtfortschrittsbalken über alle Tab-Erfolge, tab-eigene Gesamtpunkte im Kopf, Suche und Filter, Achievement-Detailansicht, Fortschrittsketten, vier neueste abgeschlossene Erfolge und eine optional eigene Heldentaten-Kategorie.

12.2 Haupt- und Unterkategorien

```
local Raids = Pack:RegisterCategory("guild_raids", {
  name = "Raids",
  tab = GuildAchievements,
})

local MoltenCore = Pack:RegisterCategory("guild_mc", {
  name = "Geschmolzener Kern",
  tab = GuildAchievements,
  parent = Raids,
})
```

Ein Erfolgs-Tab besitzt höchstens zehn Hauptkategorien. Unterkategorien zählen nicht gegen dieses Limit. Eine Kategorie und ihr parent müssen zum selben Tab gehören.

12.3 Kategorie-Balken mit automatischem Fortschritt

```
local Dungeons = Pack:RegisterTabProgressBar(
  GuildAchievements, "guild_dungeons", {
    name = "Dungeons",
    label = "Gilden-Dungeons",
    order = 10,
    tooltip = "Abgeschlossene Gilden-Dungeonerfolge",
  })
```

Für einen Erfolgs-Tab erzeugt RegisterTabProgressBar eine echte Hauptkategorie. current und maximum werden automatisch aus den verfügbaren Erfolgen dieser Kategorie einschließlich ihrer Unterkategorien berechnet. getProgress und createFrame sind hier nicht zulässig. Der Balken öffnet beim Klick die erzeugte Kategorie. Kategorien und Balken ohne verfügbare Erfolge in der Kategorie oder ihren Nachfahren bleiben unsichtbar.

12.4 Erfolge im eigenen Tab

```
Pack:RegisterAchievement("guild_first_clear", {
  category = Dungeons,
  name = "Gemeinsam siegreich",
  description = "Schließt einen Gilden-Dungeon ab.",
  points = 10,
  icon = 134400,
  criteria = {
```

```
{ key = "clear", type = "SPECIAL",
  data = { "guild_dungeon_clear" } },
},
})
```

Der Tab-Kontext wird aus der Kategorie abgeleitet. Punkte, Gesamtfortschritt, neueste Erfolge, Suche und Kategoriezähler berücksichtigen ausschließlich Erfolge dieses Tabs.

12.5 Eigene Heldentaten

```
local GuildFeats = Pack:RegisterCategory("guild_feats", {
  name = "Heldentaten",
  tab = GuildAchievements,
  featsOfStrength = true,
})

Pack:RegisterAchievement("guild_legend", {
  category = GuildFeats,
  name = "Eine Gildenlegende",
  description = "Eine nicht mehr wiederholbare Gildentat.",
  points = 0,
  icon = 134400,
  criteria = {
    { key = "legend", type = "SPECIAL",
      data = { "guild_legend" } },
  },
})
```

Pro API-Erfolgs-Tab ist genau eine Heldentaten-Hauptkategorie möglich. Sie gehört nur zu diesem Tab und beeinflusst keine Heldentaten des Hauptaddons oder anderer Erweiterungsmodule.

12.6 Exakte Limits und Zusammenspiel

Bereich	Limit
Zusätzliche Tabs insgesamt	Keine API-Grenze
Balken je leerem oder Erfolgs-Tab	10
Hauptkategorien je Erfolgs-Tab	10 insgesamt, einschließlich der durch Balken erzeugten Kategorien
Unterkategorien im Erfolgs-Tab	Keine eigene API-Grenze
Kategorien im leeren Tab	Keine API-Grenze
Heldentaten-Hauptkategorie	1 je Erfolgs-Tab

Wichtige Konsequenz

Da jeder Tab-Balken eine Hauptkategorie erzeugt, teilen sich manuell registrierte Hauptkategorien und Balken das Limit von zehn Hauptkategorien eines Erfolgs-Tabs. Zehn Balken sind möglich, wenn alle zehn Hauptkategorien über RegisterTabProgressBar entstehen.

13. Öffentliche Ereignisse

```
local token = Pack:Subscribe(API.EVENT.REGISTRATION_COMMITTED,
  function(module, moduleID, changeCount)
    if moduleID == module:GetID() then
      -- Das Modul ist vollständig finalisiert.
    end
  end)

Pack:Unsubscribe(token)
```

Ereignis	Argumente	Bedeutung
PROGRESS_CHANGED	criteriaType, data	Mindestens ein Kriterienfortschritt änderte sich
ACHIEVEMENT_COMPLETED	achievementID, ownerModuleID	Ein Erfolg wurde abgeschlossen
TITLE_CHANGED	selectedTitle oder nil	Lokale Titelauswahl wurde geändert
DATABASE_CHANGED	"batch", nil, moduleID, changes, phase	Gebündelte Struktur- oder Inhaltsänderungen
REGISTRATION_COMMITTED	moduleID, changeCount	Modulregistrierung vollständig abgeschlossen

14. Abfragen, Konstanten und Snapshots

Methode	Zweck
Pack:GetCategoryID(key)	Numerische ID einer Achievement-Kategorie
Pack:GetAchievementID(key)	Numerische Achievement-ID
Pack:GetCriteriaID(achievementKey, criteriaKey)	Numerische Kriterien-ID
Pack:GetTabHandle(key)	Handle eines registrierten leeren oder Erfolgs-Tabs
Pack:HasTab(key)	Tab-Schlüssel vorhanden
Pack:HasTabProgressBar(tab, key)	Tab-Balken vorhanden
Pack:IsAchievementCompleted(value)	Modul-eigener Abschlussstatus
Pack:GetCriteriaProgress(achievement, criteria)	Modul-eigener Kriterienfortschritt
Pack:RequestUIRefresh()	Aktive Erweiterungsansicht gebündelt aktualisieren
Pack:ResetProgress()	Nur den Fortschritt dieses Moduls zurücksetzen
API:GetRegisteredTabs()	Snapshots aller aktiven Erweiterungs-Tabs
API:GetCategory(id) / API:GetAchievement(id)	Schreibgeschützte Definitionssnapshots
API:GetEarnedTitles() / GetSelectedTitle()	Schreibgeschützte Titelsnapshots

Konstante	Wert/Bedeutung
API.TAB_TYPE.EMPTY	"empty"
API.TAB_TYPE.ACHIEVEMENTS	"achievements"
API.LIMITS.CUSTOM_TABS	Nicht vorhanden; unbegrenzt
API.LIMITS.TAB_PROGRESS_BARS	10
API.LIMITS.ACHIEVEMENT_TAB_MAIN_CATEGORIES	10
API.LIMITS.CORE_SUMMARY_PROGRESS_BARS	4

14.1 Snapshot-Erweiterungen in API 1.4

```

local category = API:GetCategory(categoryID)
print(category.tabID, category.order, category.featsOfStrength)

local achievement = API:GetAchievement(achievementID)
print(achievement.tabID, achievement.categoryID)

for _, tab in ipairs(API:GetRegisteredTabs()) do
    print(tab.key, tab.tabType, tab.tabID,
          tab.progressBarCount, tab.mainCategoryCount)
end

```

Snapshot	Neue/relevante Felder
API:GetCategory(id)	tabID, order, featsOfStrength, parentID, available
API:GetAchievement(id)	tabID, categoryID, titleReward, hideCriteriaUI, criteria
API:GetRegisteredTabs()	tabType, tabID, progressBarCount, mainCategoryCount

Alle Snapshot-Rückgaben sind Kopien beziehungsweise neue Tabellen. Sie sind zur Beobachtung und Diagnose bestimmt und erlauben keine Mutation interner Definitionen.

15. Vollständiges API-1.4-Beispiel

```

ExamplePackDB = ExamplePackDB or {}
ExamplePackDB.guild = ExamplePackDB.guild or { raids = 0 }

local API = AnniversaryAchievementsAPI
assert(API and API:IsCompatible(1, 4),
"AnniversaryAchievements Public API 1.4 required")

local Pack = API:RegisterModule("ExampleAuthor.GuildPack", {
    name = "GuildPack", version = "1.0.0",
    author = "ExampleAuthor", storage = ExamplePackDB,
})

Pack:BeginBulkRegistration()

local GUILD_EVENT = Pack:RegisterCriteriaType("GUILD_EVENT", 1)

local GuildTab = Pack:RegisterAchievementTab("guild_achievements", {
    name = "Gilde", title = "Gildenerfolge", order = 100,
})

local Dungeons = Pack:RegisterTabProgressBar(GuildTab, "dungeons", {
    name = "Dungeons", label = "Gilden-Dungeons", order = 10,
})

local Raids = Pack:RegisterTabProgressBar(GuildTab, "raids", {
    name = "Raids", label = "Gilden-Raids", order = 20,
})

local ClassicRaids = Pack:RegisterCategory("classic_raids", {
    name = "Classic-Raids", tab = GuildTab, parent = Raids,
})

Pack:RegisterAchievement("first_raid", {
    category = ClassicRaids,
    name = "Gemeinsam in den Raid",
    description = "Schließt euren ersten Gilden-Raid ab.",
    points = 10, icon = 134400,
    criteria = {
        { key = "clear", type = GUILD_EVENT,
          data = { "raid_clear" } },
    },
})

local Feats = Pack:RegisterCategory("guild_feats", {
    name = "Heldentaten", tab = GuildTab,
    featsOfStrength = true,
})

Pack:RegisterAchievement("server_first", {
    category = Feats, name = "Gildenlegende",
    description = "Vollbringt eine einmalige Gildentat.",
    points = 0, icon = 134400,
    rewardText = "Gildenlegende",
    titleReward = { key = "guild_legend",
        format = "%s, die Gildenlegende" },
    criteria = {
        { key = "legend", type = GUILD_EVENT,
          data = { "server_first" } },
    },
})

local InfoTab = Pack:RegisterTab("guild_info", {
    name = "Info", title = "Gildenübersicht", order = 110,
})

Pack:RegisterTabProgressBar(InfoTab, "raid_status", {
    name = "Raidstatus", label = "Raidstatus",
    getProgress = function()
        return ExamplePackDB.guild.raids or 0, 8
    end,
    createFrame = function(module, parent)
        local text = parent:CreateFontString(nil, "OVERLAY",
            "GameFontHighlight")
        text:SetPoint("TOPLEFT", 20, -20)
        text:SetText("Eigene Gilden-Raidübersicht")
    end,
})

Pack:EndBulkRegistration()

local function OnGuildRaidCleared()
    ExamplePackDB.guild.raids =
        (ExamplePackDB.guild.raids or 0) + 1
    Pack:Trigger(GUILD_EVENT, { "raid_clear" }, 1, "increment")
    Pack:RequestUIRefresh()
end

```

16. Grenzen, Sicherheit und Veröffentlichung

- Die Anzahl zusätzlicher Tabs ist API-seitig nicht begrenzt. Für jeden einzelnen Tab bleibt eine Breite zwischen 80 und 160 Pixeln zulässig.
- Leere Tabs: unbegrenzt viele Haupt- und Unterkategorien, aber höchstens zehn Tab-Fortschrittsbalken.
- Erfolgs-Tabs: höchstens zehn Hauptkategorien und zehn Balken; jeder Balken ist zugleich eine Hauptkategorie. Unterkategorien besitzen keine eigene API-Grenze.
- Pro Erfolgs-Tab ist genau eine eigene Heldentaten-Hauptkategorie möglich. Die native Heldentaten-Kategorie bleibt geschützt.
- Callbacks werden geschützt aufgerufen, sollten aber keine langen Schleifen ausführen. Teure Werte in eigenen SavedVariables oder Caches vorberechnen.
- Keine internen ns-Tabellen, CA_* SavedVariables, AchievementFrame-Funktionen oder Datenbankobjekte direkt verändern.
- Module und Definitionen werden während derselben Sitzung nicht entladen oder neu registriert. Schlüssel und IDs nach Veröffentlichung stabil halten.
- Vor Veröffentlichung mit frischen und bestehenden SavedVariables, mehreren parallelen Erweiterungsmodulen und beiden unterstützten Client-Flavors testen.

Ziel von API 1.4

Gilden, Community-Projekte und Entwickler können vollständig eigene Erfolgsbereiche bereitstellen, ohne die internen UI- oder Datenbankstrukturen des Hauptaddons zu patchen. Das Hauptaddon bleibt Eigentümer der Darstellung und Validierung; das Erweiterungspack bleibt Eigentümer seiner Definitionen und seines persistenten Fortschritts.

Ende der Public-API-1.4-Dokumentation